

SQL

You have a `transactions` table:

`transactions`

column

`txn_id`

`account_id`

`txn_ts`

`amount`

`merchant`

Write a SQL query to identify:

1. Each customer's first transaction
2. Previous transaction amount
3. Percentage increase from previous transaction
4. Flag transactions where spend increased by more than 200%

Option1:

```
with _row_numbers as (
    SELECT txn_id, account_id, txn_ts, amount, merchant
           ROW_NUMBER() OVER (PARTITION BY account_id ORDER BY txn_ts ASC )
as row_num_asc,
           ROW_NUMBER() OVER (PARTITION BY account_id ORDER BY txn_ts DESC )
as row_num_desc
    FROM transactions
),

_first_transaction as (
    SELECT account_id, txn_id as first_txn_id
    FROM _row_numbers
    WHERE row_num_asc = 1
),

_last_transaction as (
    SELECT account_id, txn_id as last_txn_id, amount as last_txn_amount
```

```

        WHERE _row_numbers
        WHERE row_num_desc = 1
    ),

    _second_last_transaction as (
        SELECT account_id, txn_id as second_last_txn_id, amount as second_last_txn_amount
        WHERE _row_numbers
        WHERE row_num_desc = 2
    ),

    _final as (
        SELECT a.account_id, a.first_txn_id, b.last_txn_amount,
        COALESCE(c.second_last_txn_amount, 0) as second_last_txn_amount,
            div0(b.last_txn_amount - c.second_last_txn_amount,
        c.second_last_txn_amount)*100.0 as pct_change,
            CASE WHEN pct_change > 200.00 then 1 ELSE 0 END AS flag_pct_increase
        FROM _first_transaction a
        LEFT JOIN _last_transaction b ON a.account_id = b.account_id
        LEFT JOIN _second_last_transaction c ON a.account_id = c.account_id
    )

SELECT * FROM _final

```

Option 2:

```

with _row_numbers as (
    SELECT txn_id, account_id, txn_ts, amount, merchant
            ROW_NUMBER() OVER (PARTITION BY account_id ORDER BY txn_ts ASC )
as row_num_asc,
            ROW_NUMBER() OVER (PARTITION BY account_id ORDER BY txn_ts DESC )
as row_num_desc,
            LEAD(amount) OVER (PARTITION BY account_id ORDER BY txn_ts DESC ) as
second_last_txn_amount
    FROM transactions
),

_first_transaction as (
    SELECT account_id, txn_id as first_txn_id
    FROM _row_numbers
    WHERE row_num_asc = 1
),

_last_transaction as (

```

```

        SELECT account_id, txn_id as last_txn_id, amount as last_txn_amount,
second_last_txn_amount
        WHERE _row_numbers
        WHERE row_num_desc = 1
    ),

    _final as (
        SELECT a.account_id, a.first_txn_id, b.last_txn_amount,
COALESCE(b.second_last_txn_amount, 0) as second_last_txn_amount,
        div0(b.last_txn_amount - b.second_last_txn_amount,
b.second_last_txn_amount)*100.0 as pct_change,
        CASE WHEN pct_change > 200.00 then 1 ELSE 0 END AS flag_pct_increase
        FROM _first_transaction a
        LEFT JOIN _last_transaction b ON a.account_id = b.account_id
        LEFT JOIN _second_last_transaction c ON a.account_id = c.account_id
    )

SELECT * FROM _final

```

- If txn_id is integer we can use it along with ts

Problem Statement

You are given 3 source tables, Assume all tables have created_at and updated_at column.

transactions

column	type
id	bigint
uuid	string
amount	decimal

transaction_status

column	type
transaction_id	bigint
status	string

user_details

column	type
uuid	string
user_name	string

You need to design and implement a Spark/dbt job to create an **incremental fact table** by joining these tables.

Requirements

1. The pipeline should process data incrementally.
2. Existing records should be updated if transaction status changes.
3. The pipeline should be idempotent.
4. The solution should scale for large datasets.

```
{{
    materialized = 'table',
    query_tag = 'fct_transactions_users',
    incremental_strategy = 'merge',
```

```

        pre_hook = '{{a delete macro which deletes the data which was there in last 1.5 hours}}'
--frequency: 30 mins
}}

```

{% set execution_date = get_execution_date() %} -- Write this somewhere in macros and this gets updated everytime on dbt run 30 mins incrementally

```

with _changes_in_transaction_status as (
    SELECT transaction_id, status , created_at as status_createdat, updated_at as
status_updatedat from transaction_status
    WHERE 1=1
    AND {{ if_incremental() }}
        updated_at >= datediff('minutes', 90, execution_date) -- Basically want to check
for last 90 mins based on execution_date
    ),

```

```

_changes_in_users as (
    SELECT uuid, user_name, created_at as user_createdat, updated_at as
user_updatedat from user_details
    WHERE 1=1
    AND {{ if_incremental() }}
        updated_at >= datediff('minutes', 90, execution_date) -- Basically want to check
for last 90 mins based on execution_date
    ),

```

```

_transactions_based_on_status as (
    SELECT id, uuid, amount, created_at, updated_at from transaction WHERE id in
(SELECT transaction_id from _changes_in_transaction_status)
    ),

```

```

_transactions_based_on_user as (
    SELECT id, uuid, amount, created_at, updated_at from transaction
    WHERE 1=1
    AND updated_at >= datediff('minutes', 90, execution_date)
    uuid in (SELECT uuid from _changes_in_users)
    ),

```

```

_combined_txn_changes as (
    SELECT * FROM _transactions_based_on_status
    UNION
    SELECT * FROM _transactions_based_on_user
    ),

```

```

_final as (
    SELECT b.transaction_id, a.uuid, a.amount, a.created_at as txn_created_at, b.status as
status, b.status_createdat, b.status_updatedat, c.user_name, c.user_createdat,
c.user_updatedat
    FROM _combined a
    LEFT JOIN _changes_in_transaction_status b ON a.id = b.transaction_id
    LEFT JOIN _changes_in_users c ON a.uuid = c.uuid
),

casting as (
    SELECT transaction_id::BIGINT as transaction_id,
    .....
)

SELECT * FROM casting

```

1. the changes rows in a df
2. copy to a staging table (temp table)
3. DELETE using txn_id from main table using this temp table
4. INSERT the data from temp to main
5. DELETE data from temp table

3rd step happened -> queried main table -> you get a result -> INSERT happened -> you get a different result

freshness and volumes they should be intact: yml file
dq tests: not null, unique etc.

fct_table: not null, unique,
datepart: for the last 90 days there should be data for everyday
ranges:
-- z-score: custom macro

severity: error or warn